

Computational Macroeconomics

Lecture 1: Introduction

Kanato Nakakuni

September 7, 2026

Course objectives

1. Introduce computational methods, techniques, and procedures for solving and analyzing macroeconomic models
2. Develop practical skills in computationally solving macroeconomic models through hands-on exercises

We will achieve these goals by working mainly on:

- **Overlapping generations (OLG)/Lifecycle model**
 - One of the core frameworks in macroeconomics
 - Especially useful when interested in topics related to demographics (population aging, social security) and household behavior over the lifecycle (human capital accumulation, fertility, marriage/divorce, retirement, education investment, etc.)
- Its extension incorporating household **heterogeneity** (income, family structure, etc.)

Course structure

- Part 1: Solving household dynamic optimization problems (Lectures 2 and 3)
 - Keyword: discretization, grid search, endogenous grid methods, backward induction
- Part 2: Solving for equilibrium (Lectures 4–7)
 - Keyword: stationary equilibrium, transitional dynamics, calibration, counterfactual analysis
 - Application to macro implications of social security reforms
- Part 3: Notes on other important methods (Lectures 8–10)
 - Keyword: value (policy) function iteration, root finding, optimization, interpolation
- Part 4: Advanced topics and application (Lectures 11–14)
 - Keyword: Aiyagari model, discretizing stochastic income process, heterogeneity
 - Application to macro effects of education policy

Basic information

- **Period:** September 7, 2026 – December 7, 2026 (+ final exam)
- **Time:** Monday 13:45–15:15
- **Place:** 158 PC pool (L7, 3-5)
- **Prerequisite:** Intermediate-level courses in micro and macro
- **Grading:** Assignments (50%) + Final exam (50%, 90 min)
 - Tentative plan: A coding assignment will be posted in mid-October, with a deadline of December 7, 2026.

Basic information (cont'd)

- **Instructor:** Kanato Nakakuni (kanato.nakakuni@uni-mannheim.de)
- **Course materials:** Provided on my GitHub repository, *ComputationalMacro*
 - Please send me an email today with your name and email address. In reply, I will send access invitation to the GitHub repository

Programing language

- In explaining sample code, I will use Julia
- You may use other programming language (e.g., MATLAB, Fortran, Python) for in-class exercises and to submit your coding assignments if that is more comfortable
- Otherwise, please install Julia and a suitable code editor (e.g., Visual Studio Code) on your own laptop
- Please install them before the next class. Links for installation:
 - Julia: <https://julialang.org/install/>
 - VS code: <https://code.visualstudio.com/>

AI is changing how we do computation

- Rapid advances in AI, with more coding-focused tools (e.g., Claude Code, Copilot, Codex)
 - Write, debug, explain, improve code, etc.
- These can substantially increase coding productivity
 - Learning how to use them effectively is valuable
 - See, e.g., “AI Agents for Economic Research” by Aniket Panjwani
<https://www.youtube.com/watch?v=YPv9BqweQIo>

Why learn computation if AI can already code

- Modeling (theory) and computation (implementation) are interconnected
 - Too many state variables → computational infeasibility
 - Estimation may require solving the model thousands of times → computational tractability shapes model design
 - Analytically convenient assumptions reduce computational burden (e.g., preference shocks follow iid EV type-I)
- You need to know whether the output is correct and what to ask AI to do
 - What algorithms are available to overcome computational bottlenecks (e.g., speed, accuracy)?
 - Use AI as an assistant, not an authority